A Classical Reasoning Agent for Natural-Language Entailment

Classic Logic Engines and a Translator We Trained

Aniruddha Bohra
Student ID: p25cs0201
AI Assignment 2026

***Abstract*—This report describes a program that reads a short English text of facts and rules and decides whether a given statement is true (it follows from the text) or false. The program first turns the English into logic. Then a classic reasoning method gives the answer. We use three methods: forward chaining for rule puzzles (RuleTaker), cause-and-effect reasoning for "what if" questions (CounterBench), and resolution for first-order logic (FOLIO). On 153,315 public questions the program is correct 99.96% of the time and needs only 16.9 seconds in total. It answers all 151,911 RuleTaker test questions correctly. Some FOLIO problems come only as free English. For these we built two translators: a rule-based one (64.7% correct) and a small model we trained ourselves (71.1% correct). A generative model never gives the final answer; the logic engine always decides. We also tested the program on 10,600 new problems we generated, and it answered all of them correctly.**

***Index Terms*—logical reasoning, forward chaining, cause and effect, resolution, natural language**

I. Introduction

Some benchmarks test reasoning with English text. Each problem has a context (facts and rules) and a statement. The task is to say whether the statement follows from the context. We use three public benchmarks: RuleTaker [1], FOLIO [2] and CounterBench [3].

The assignment has one main rule: the final answer must come from classic AI reasoning [4]. We may turn the English into logic in any way we like. So our program works in four steps: English → logic → reasoning → True/False (Fig. 1).

Our main contributions are: (i) a router that sends each problem to one of three reasoning engines; (ii) a fix for rules that contain "not" in a loop (the well-founded semantics [5]), which raised RuleTaker accuracy from 98.60% to 100%; (iii) two ways to translate free English into logic, one rule-based and one trained by us; and (iv) a fair testing process: we never used test data to make design choices, and we also report ideas that did not work.

II. How We Represent the Problems

Rule puzzles (RuleTaker). A fact is a property or a relation, for example *young(bob)* for "Bob is young" and *like(cat, dog)* for "The cat likes the dog". A fact can be positive or negative, so "Bob is not big" is stored directly. A rule has conditions and a result, where $?x$ stands for any person or thing: "If someone is smart and kind then they are successful" becomes $smart(?x) \wedge kind(?x) \rightarrow successful(?x)$. Our parser understands if-then rules, general rules such as "Rough, red people are big", relations, words like "can", pronouns ("they", "it") and short forms such as "white and not smart".

Cause and effect (CounterBench). Each event gets a true/false formula: "not Frim and Trune causes Qado" becomes $qado := \neg frim \wedge trune$. If an event has several causes, any one of them is enough (OR). From the question we read the imagined change ("if not X"), what really happened ("instead of X"), what was observed, and any given probabilities.

First-order logic (FOLIO). Formulas may use $\forall$ (for all), $\exists$ (there exists), $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$ and $\oplus$ (either-or). We turn every formula into a list of simple "clauses", the form that the resolution method needs.

III. How the Program Reasons

Forward chaining. The program starts from the facts and applies the rules again and again to find new facts, until nothing new appears. To be fast, each rule is stored under the words in its conditions, so a new fact only checks the rules that can use it. For each text, all facts are worked out once and saved, so further questions about the same text are quick lookups. The program also remembers which rule produced each fact, so it can explain its answer.

Rules with "not". RuleTaker uses the closed-world idea: what cannot be proved is false. Usually the rules can be ordered so that each "not" is checked only after its fact is final. But in about one fifth of the questions the rules form a loop through "not" (for example, *blue* depends on *not round*, and *round* on *not blue*). For these we use the *well-founded semantics* [5]: the program keeps two lists, "surely true" and "maybe true", and updates them in turn until both stop changing. Anything still unsure is answered False.

"What if" questions. We follow Pearl's three steps [6]: (1) find the starting values that match what was observed; (2) force the imagined change, for example set X to false; (3) compute all other events again. We use three values

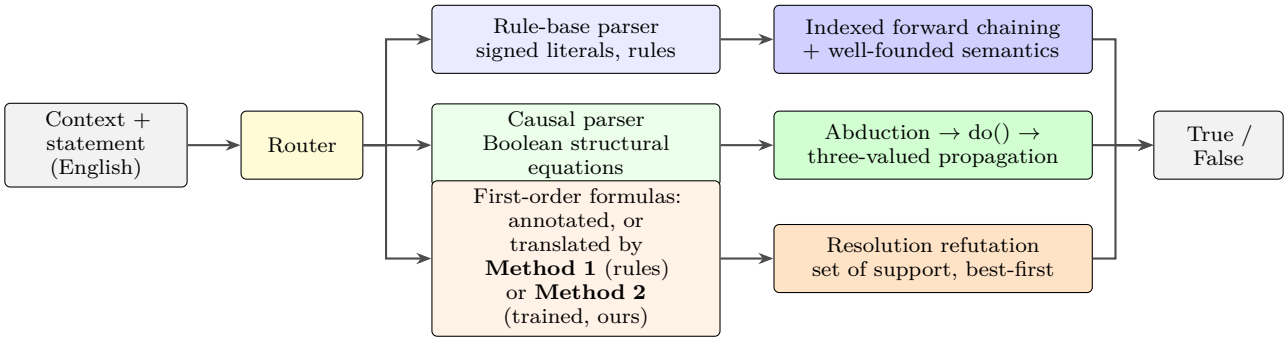


Fig. 1. How the program works. The router looks at each problem and sends it to the right engine. Every answer comes from one of the three logic engines.

(true, false, unknown), so loops in the cause graph do not stop the program when the answer is already clear.

Resolution. To check a statement, the program adds its opposite to the facts and searches for a contradiction [7]. If it finds one, the statement is true. The search always involves the opposite statement ("set of support" [8]), tries the shortest clauses first, and stops after a fixed number of steps.

Backward chaining. Starts from the question and works back to the facts; used only for comparison.

IV. Two Ways to Translate Free English

Some FOLIO problems come only as English, without logic formulas. We built two translators. Both write logic for the same resolution engine (Table I). One setting in the notebook chooses between them: `ENGLISH_METHOD = "rule"` or `"trained"`.

Method 1: rule-based. Hand-written sentence patterns, such as "All X are Y", "Some X are Y" and "If … then …", are turned into formulas. It is instant and needs only Python.

Method 2: trained translator (ours). We fine-tuned flan-t5-base [9], [10], a model with 250 million parameters, to translate a whole problem at once. Seeing all sentences together helps the model use the same name for the same idea. We trained it only on FOLIO's training data: 299 stories, turned into 1,461 examples, with 33 more stories kept aside to choose the best version. The model's vocabulary has no logic symbols, so it writes words such as FORALL and XOR, which we turn back into symbols. Training took 6.5 minutes on one GPU. Every translation is checked: all formulas must be readable, and they must not contradict each other. If the check fails, or the model is not available, the program uses Method 1 instead. The training code is in the notebook as an appendix.

Default. Method 1 is the default: the score counts all questions together and speed is graded, and English-only FOLIO problems are few, so Method 2 adds little overall accuracy but can slow a run a lot.

TABLE I
The Two Translation Methods

	Method 1: rule-based	**Method 2: trained (ours)**
How	Sentence patterns → formulas	flan-t5-base, whole problem in one pass
Needs	Python only; offline	torch, transformers, model
Speed	<1 ms per problem	0.8 s (T4 GPU), 1.3 s (CPU)
Accuracy	64.71%	71.08% (GPU), 71.57% (CPU)
Setting	`"rule"` (default)	`"trained"`

TABLE II
Main Results (One Complete Run on Google Colab)

Dataset (split)	Queries	Acc.	Time	ms/q
RuleTaker (test)	151,911	100.00%	11.89 s	0.078
CounterBench (all)	1,200	95.00%	1.06 s	0.885
FOLIO (val., formulas)	204	98.04%	3.95 s	19.3
Overall	**153,315**	**99.96%**	**16.90 s**	**0.110**
FOLIO English, Method 1	204	64.71%	0.15 s	0.7
FOLIO English, Method 2	204	71.08%	162 s†	796

†translation on a Colab T4 GPU. English-only rows are not part of the overall figure.

V. How We Tested

We made all design choices with practice data only, never with test data. For RuleTaker we chose settings on the dev set (75,872 questions) and report results on the test set (151,911). For FOLIO we used the 1,004 training problems to train Method 2 and report results on the 204 validation problems. CounterBench has only one set (1,200 questions), so we chose settings on one half and checked them on the other half. Times include reading and reasoning, but not downloads. The main run was on Google Colab (Table II).

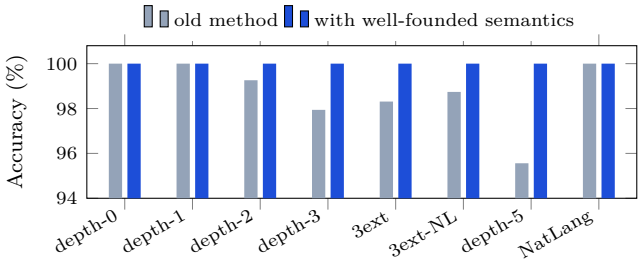


Fig. 2. RuleTaker test accuracy by depth (151,911 questions). Mistakes grew with depth until we added the well-founded semantics.

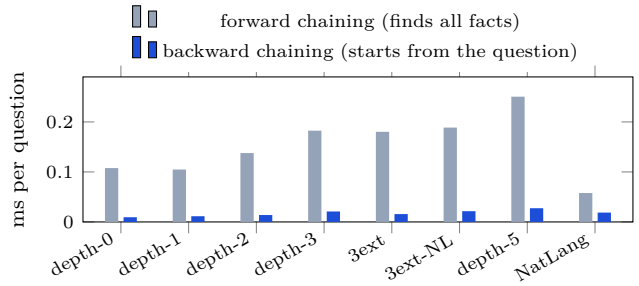


Fig. 3. Time per question when nothing is saved between questions (1,000 questions per group, reading time not included). Both methods are 100% correct. Rules used per question: 6.6 (forward) and 1.2 (backward).

VI. Results

A. Rules with "not" in a loop

Our first full run scored 98.60%. All mistakes were in the deeper questions (Fig. 2), which pointed to a reasoning problem, not a reading problem. We split 30,000 practice questions into two groups. Without loops (23,838 questions) the old method was perfect; with "not" loops (6,162 questions) it was right only 90.85% of the time. With the well-founded semantics, every question was answered correctly.

B. Forward versus backward chaining

When each question is answered alone, backward chaining is 8.7 times faster (0.017 vs 0.151 ms) and uses 5.5 times fewer rules, because it only looks at rules that matter for the question (Fig. 3). In real use, however, each text has many questions and forward chaining saves its results, so both took about the same total time (10.35 and 11.71 s forward, 10.56 and 10.73 s backward, in two runs), both at 100%. Our new tests (Section VI-F) showed that backward chaining can get "not" loops wrong, so we keep forward chaining.

C. CounterBench

All basic questions are correct. At first, five questions crashed because their cause graph has a loop; using three values fixed this. For the remaining mistakes we tested other ways of reading the questions, six combinations in total

TABLE III
CounterBench Accuracy for Each Way of Reading the Questions (%)

	change forced			change ignored		
Type (n)	all	obs.	prior	all	obs.	prior
Basic (250)	**100.0**	100.0	100.0	100.0	100.0	100.0
Joint (250)	**96.0**	96.0	96.0	90.0	90.0	90.0
Nested (250)	**96.0**	96.0	96.0	76.0	76.0	76.0
Conditional (250)	**92.0**	92.0	92.0	92.0	92.0	92.0
Backdoor (200)	**90.0**	87.5	75.0	90.0	87.5	75.0

all/obs./prior: use all evidence, only observations, or only the given probability. Bold: our choice (standard). Overall 95.00%.

(Table III). The standard way was best or equal in every row. Choosing on one half of the data and testing on the other gave 95.00% both times. The mistakes come in groups of five, because each question appears with five different made-up names, so only about 12 different questions are wrong. In two of them, the observed fact is impossible under the given rules. Human experts score 97.75% [3]. Adding special rules just for these questions would be overfitting.

D. FOLIO with formulas

When the logic formulas are given, the resolution engine is correct 98.04% of the time. Fixing how we read the formulas (special letters, the $\longleftrightarrow$ symbol and missing brackets) raised this from 96.57%. Trying the shortest clause first took 0.92 s for the 204 problems, against 2.09 s for the simple order, with the same answers. A problem that is not true makes the engine search until the step limit, so the limit decides the time. Every proof we found needed at most 18 steps (usually about 5). Lowering the limit from 400 to 100 made the engine 20 times faster and did not change a single answer (Fig. 5).

E. Free English: Method 1 versus Method 2

We improved Method 1 by studying the training sentences it could not read. It then translated 89% of the validation sentences (up from 66%) and 126 of the 204 whole problems (up from 41). Yet accuracy rose only from 64.22% to 64.71%, the same as always answering False. The reason: the same idea is written in different words in different sentences, so the proofs do not connect. Joining names that share words made it worse (62.25%), because shared words cannot tell "aware" from "unaware".

Method 2 handles this better and reaches 71.08% (Fig. 6). It translated 188 of the 204 problems, with 19 correct proofs and 6 wrong ones (Method 1: 3 and 2), and it rejected 8 translations that contradicted themselves. We also tried several changes, judged on 111 practice questions: beam search (no better, twice as slow), 8-bit compression (less accurate), extra copies with swapped names (63.96% and 70.27% against 67.57%, just random variation), the last training round (66.67%), and a bigger model, flan-t5-large (71.17%, but about three times slower). We kept none of them.

(a) Forward chaining (assignment example)

```
Facts : young(bob), kind(bob) Rule : young(?x) → smart(?x) Rule
: smart(?x) ∧ kind(?x) → successful(?x) successful(bob) <= "If
someone is smart and kind, ..." smart(bob) <= "If someone is
young, then ..." young(bob) (given fact) kind(bob) (given fact)
ANSWER: True [3 rule firings]
```

(b) Negation as failure (RuleTaker, depth 1)

```
Rule : white(?x) ∧ NOT smart(?x) → cold(?x) cold(harry) <= "If
something is white and not smart ..." white(harry) (given fact)
NOT smart(harry) (not derivable) ANSWER: True
```

(c) Counterfactual on a cyclic graph (CounterBench)

```
blaf := NOT ziklo trune := ziklo sline := blaf AND trune AND vork
vork := NOT (trune OR sline) observed blaf=1, factual ziklo=1,
do(ziklo=0) world: trune 0 ⇒ sline 0 ⇒ vork 1 ⇒ lumbo 1
ANSWER: True (gold: True)
```

(d) Resolution refutation (FOLIO, annotated formulas)

```
∀x (Drinks(x,coffee) → Dependent(x,caffeine)) ∀x
(Drinks(x,coffee) ⊕ ¬Wants(x,caffeine)) ... Goal:
¬Wants(rina,caffeine) ∨ ¬Aware(rina,caffeine) negated goal:
{Wants(rina,..)}, {Aware(rina,..)} empty clause after 6
given-clause steps ANSWER: True
```

Fig. 4. Example reasoning traces: (a) forward chaining, (b) a rule with "not, (c) a "what if question with a loop, (d) resolution. In (d) some names are shortened to save space.

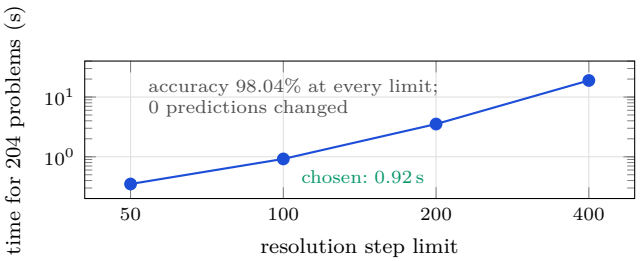


Fig. 5. Step limit versus time on FOLIO (Colab). The longest proof needs only 18 steps, so a limit of 100 saves time without changing any answer.

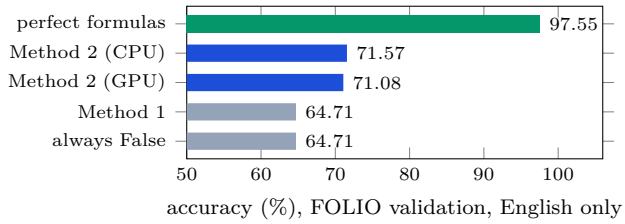


Fig. 6. Accuracy on free English. Method 2 gets 13–14 more problems right out of 204. "Perfect formulas is the best score possible with a perfect translator.

F. Tests on new problems

The hidden test uses new texts, so we made new problems ourselves and checked them with a separate solver that shares no code with our program. The rule puzzles use words that RuleTaker never uses ("Zara is brave", "The grizzly bear fears the owl"), or say every rule in a new way ("Every brave person is calm", "Anyone who is brave is also calm"). The "what if" questions use new names and random cause graphs (Table IV). These tests found two problems, and we fixed both: rules about "something" were not applied to a person named only in the question, and backward chaining could get "not" loops wrong.

VII. Reasoning Traces

Fig. 4 shows example traces, printed from the rules the program used.

TABLE IV
New Problems, Checked by a Separate Solver

Generated test set	Questions	Accuracy
Rule bases, new vocabulary (two seeds)	6,000	100.00%
Rule bases, every rule rephrased	3,000	100.00%
Causal questions, new names, 4 types	1,600	100.00%

VIII. Conclusion

A few classic reasoning engines, each used for the right kind of problem, solve the public benchmarks almost completely: 100% on RuleTaker, 98.04% on FOLIO with formulas, and 95% on CounterBench, where the remaining mistakes follow no clear rule. The hardest part is translating free English. Our trained translator raises FOLIO from 64.7% to 71.1%, but correct formulas would allow 97.6%. The main reason is that only 299 training stories were available. A bigger model (flan-t5-large got four more practice questions right, but is about three times slower) and more checked training data are the clearest next steps. We could not test a model trained on code (CodeT5+), because its files do not work with our library version.

References

[1] P. Clark, O. Tafjord, and K. Richardson, "Transformers as soft reasoners over language," in *Proc. IJCAI*, 2020.

[2] S. Han *et al.*, "FOLIO: Natural language reasoning with first-order logic," arXiv preprint arXiv:2209.00840, 2022.

[3] "CounterBench: A benchmark for counterfactuals reasoning in large language models," arXiv preprint arXiv:2502.11008, 2025.

[4] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.

[5] A. Van Gelder, K. A. Ross, and J. S. Schlipf, "The well-founded semantics for general logic programs," *J. ACM*, vol. 38, no. 3, pp. 619–649, 1991.

[6] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge University Press, 2009.

[7] J. A. Robinson, "A machine-oriented logic based on the resolution principle," *J. ACM*, vol. 12, no. 1, pp. 23–41, 1965.

[8] L. Wos, G. A. Robinson, and D. F. Carson, "Efficiency and completeness of the set of support strategy in theorem proving," *J. ACM*, vol. 12, no. 4, pp. 536–541, 1965.

[9] C. Raffel *et al.*, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.

[10] H. W. Chung *et al.*, "Scaling instruction-finetuned language models," arXiv preprint arXiv:2210.11416, 2022.